

Day 5 — Trees & Binary Search Trees

Faculty: Vipul Tembulwar

Terminology & types

Traversals (BFS, DFS, pre/in/post)

Complete binary tree in an array

BST create / search / delete

Everything so far has been **linear**: one element after another. Real data is often **hierarchical**: folders inside folders, a company org chart, an HTML page, a family tree. Trees represent that. Binary search trees also give us $O(\log n)$ search, insert and delete with keys kept in sorted order, which is how **TreeMap** works.

1. Introduction to trees

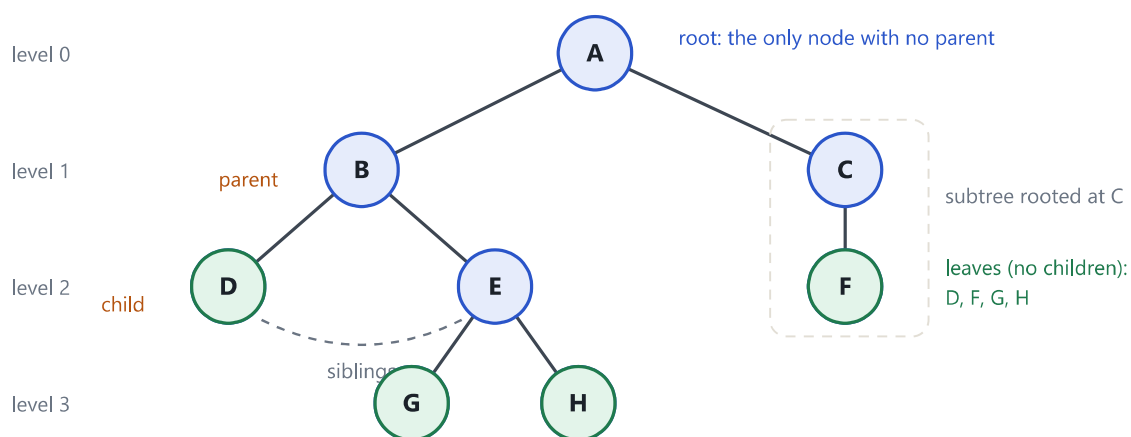
DEFINITION Tree

A collection of **nodes** connected by **edges**, with one special node called the **root**, where every other node has **exactly one parent** and there are **no cycles**. A tree with n nodes has exactly $n - 1$ edges.

Trees you use every day:

- **File system:** `C:\` → Users → Vipul → Desktop → ...
- **HTML DOM:** `<html>` → `<body>` → `<div>` → `<p>`
- **Java class hierarchy:** `Object` → `AbstractCollection` → `AbstractList` → `ArrayList`
- **Expression tree:** `(3 + 4) * 2` has `*` at the root, with `+` and `2` as its children
- **Decisions:** the subsets tree from Day 4, and game trees in chess engines

2. Tree terminology



height of tree = longest root-to-leaf path = 3 edges ($A \rightarrow B \rightarrow E \rightarrow G$) · depth of E = 2

A tree: nodes connected by edges, with one root and no cycles

Term	Meaning
Root	the top node, with no parent
Parent / child	an edge from a node down to its child
Siblings	nodes with the same parent
Leaf (external node)	a node with no children
Internal node	a node with at least one child
Subtree	a node together with all its descendants
Ancestor / descendant	any node on the path up to the root, or down from a node
Depth of a node	the number of edges from the root to it. The root has depth 0
Level	all nodes with the same depth
Height of a node	the number of edges on the longest path down to a leaf. A leaf has height 0
Height of the tree	the height of the root. We use -1 for an empty tree
Degree	the number of children of a node

NOTE Some books count height in nodes, not edges

In those books, a single node has height 1 and an empty tree has height 0. We count **edges** throughout, as `TreeMap` and most algorithm books do. Always check which convention a question uses.

3. Binary trees

DEFINITION Binary tree

A tree in which every node has **at most two** children, called the **left** child and the **right** child. The order matters: a node with only a left child is different from a node with only a right child.

JAVA `BinaryTreeBasics.java`

```
static class Node {
    char data;
    Node left, right;          // null = no child
    Node(char data) { this.data = data; }
}
```

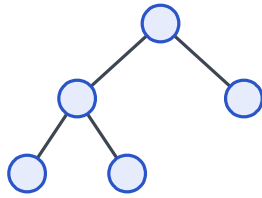
A binary tree is **recursive**: it is either empty (`null`), or a root with a left binary tree and a right binary tree. So most tree methods are three lines long: handle `null`, recurse left, recurse right.

JAVA

```
static int size(Node n) { return n == null ? 0 : 1 + size(n.left) + size(n.right); }
static int height(Node n) { return n == null ? -1 : 1 + Math.max(height(n.left), height(n.right)); }
static int leaves(Node n) {
    if (n == null) return 0;
    if (n.left == null && n.right == null) return 1;
    return leaves(n.left) + leaves(n.right);
}
```

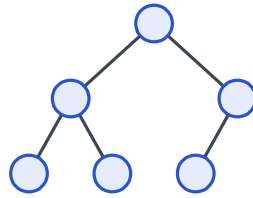
Each visits every node once: **$O(n)$ time**, and **$O(h)$ stack space**, where h is the height.

Types of binary trees



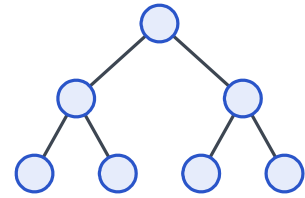
Full

0 or 2 children each



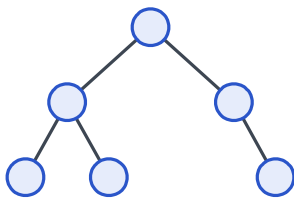
Complete

last level filled left → right



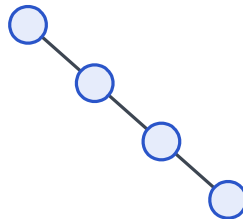
Perfect

all leaves on one level



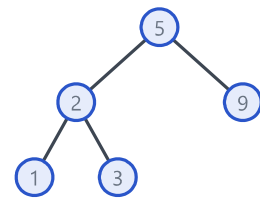
Balanced

heights differ by ≤ 1



Degenerate

one child each: a list



Binary search tree

left < node < right

The same words appear in heaps (complete), AVL trees (balanced) and worst-case analysis (degenerate)

Type	Rule	Why it matters
Full (strict)	every node has 0 or 2 children	expression trees; leaves = internal nodes + 1
Complete (almost complete, ACBT)	all levels full except possibly the last, which fills left to right	can be stored in an array with no gaps: heaps, PriorityQueue
Perfect	every internal node has 2 children and all leaves are at the same depth	has exactly $2^{h+1} - 1$ nodes
Balanced	at every node, the left and right subtree heights differ by at most 1	height stays $O(\log n)$: AVL trees (Day 6)
Degenerate (skewed)	every parent has one child	really a linked list, with height $n - 1$

Type	Rule	Why it matters
Binary search tree	left subtree < node < right subtree, at every node	search in $O(h)$

`TreeTypesCheck.java` tests each property recursively:

TEXT

```
perfect                full=true  complete=true  perfect=true  balanced=t
complete (last level partial) full=false complete=true  perfect=false balanced=t
full but not complete  full=true  complete=false perfect=false balanced=t
degenerate (linked list) full=false complete=false perfect=false balanced=f
looks like BST, is not  full=false complete=false perfect=false balanced=t
```

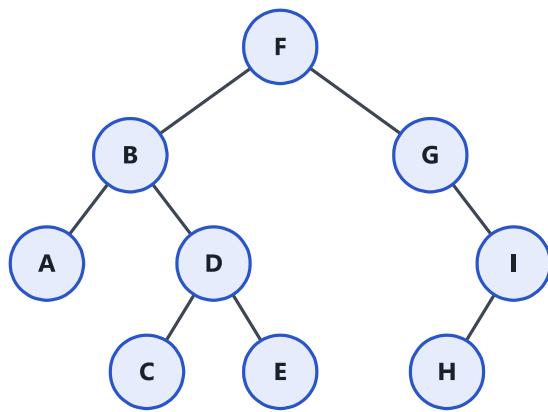
QUICK
CHECK

What is the maximum number of nodes in a binary tree of height h ? And the minimum height of a binary tree with n nodes?

Show answer

4. Tree traversals

A **traversal** visits every node exactly once. For a list there is one obvious order. For a tree there are four standard ones.



pre-order Root · Left · Right

F B A D C E G I H

in-order Left · Root · Right

A B C D E F G H I

post-order Left · Right · Root

A C E D B H I G F

level-order level by level (queue)

F B G A D I C E H

In-order of a BST gives the keys in sorted order. Here the letters come out as A ... I.

Depth-first traversals differ only in WHEN the root is visited. Level order needs a queue.

Depth-first: pre-order, in-order, post-order

JAVA

```

static void preorder(Node n) {           // Root, Left, Right
    if (n == null) return;
    System.out.print(n.data + " ");
    preorder(n.left);
    preorder(n.right);
}

static void inorder(Node n) {           // Left, Root, Right
    if (n == null) return;
    inorder(n.left);
    System.out.print(n.data + " ");
    inorder(n.right);
}

static void postorder(Node n) {         // Left, Right, Root
    if (n == null) return;
    postorder(n.left);
    postorder(n.right);
}
  
```

```
System.out.print(n.data + " ");
}
```

Traversal	Typical use
Pre-order	copy or serialise a tree (the root comes first); print a folder listing
In-order	get a BST's keys in sorted order
Post-order	delete or free a tree (children before the parent); compute folder sizes; evaluate an expression tree
Level-order	print level by level; find the shortest path in an unweighted tree; build a heap

Breadth-first: level-order uses a queue

JAVA

```
static void levelOrder(Node root) {
    Queue<Node> q = new ArrayDeque<>();
    q.add(root);
    while (!q.isEmpty()) {
        Node n = q.poll();           // take the oldest node
        System.out.print(n.data + " ");
        if (n.left != null) q.add(n.left);    // its children wait behind
        if (n.right != null) q.add(n.right);  // everything already queued
    }
}
```

The **queue** (FIFO) from Day 2 is exactly what makes nodes come out level by level. Swap it for a **stack** (LIFO) and you get depth-first order instead. The same idea gives BFS and DFS on graphs (Day 9).

Depth-first without recursion (syllabus lab: DFS)

Recursion uses the JVM call stack. We can manage our own stack instead, which never overflows on a deep tree:

JAVA IterativeTraversals.java

```

static void preorder(Node root) {
    Deque<Node> st = new ArrayDeque<>();
    st.push(root);
    while (!st.isEmpty()) {
        Node n = st.pop();
        System.out.print(n.data + " ");
        if (n.right != null) st.push(n.right);    // push RIGHT first ...
        if (n.left != null) st.push(n.left);      // ... so LEFT is popped first
    }
}

static void inorder(Node root) {
    Deque<Node> st = new ArrayDeque<>();
    Node cur = root;
    while (cur != null || !st.isEmpty()) {
        while (cur != null) { st.push(cur); cur = cur.left; }    // go all the way left
        cur = st.pop();
        System.out.print(cur.data + " ");                        // visit
        cur = cur.right;                                         // then the right subtree
    }
}

```

The iterative in-order on a **10-lakh-deep** degenerate tree runs in about 40 ms. The recursive version would crash with `StackOverflowError`.

**QUICK
CHECK**

A tree's pre-order is **F B A D C E G I H** and its in-order is **A B C D E F G H I**. Which node is the root, and which nodes are in its left subtree?

Show answer

5. Complete binary tree in an array (ACBT)

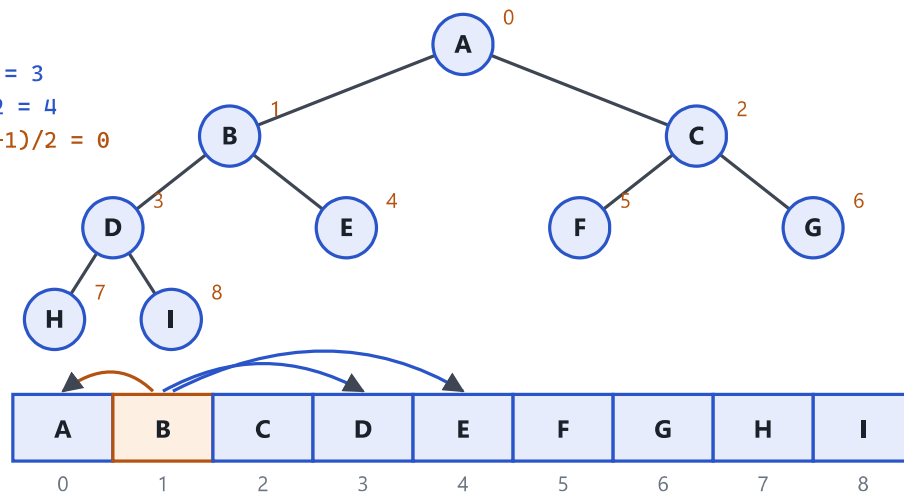
Number the nodes of a **complete** binary tree level by level, left to right, starting at 0. There are no gaps, so the number can be an **array index**, and references are not needed at all.

$i = 1$ (B)

$\text{left} = 2i + 1 = 3$

$\text{right} = 2i + 2 = 4$

$\text{parent} = (i - 1) / 2 = 0$



Level-order numbering leaves no gaps in a complete tree, so positions replace references. PriorityQueue stores its heap like this.

For the node at index i	Formula
left child	$2i + 1$
right child	$2i + 2$
parent	$(i - 1) / 2$ (integer division)
is a leaf	$2i + 1 \geq \text{size}$
height of the tree	$\lceil \log_2 \text{size} \rceil$

JAVA `ArrayBinaryTree.java`

```

void preorder(int i) {
    if (i ≥ size) return;           // outside the array = null child
    System.out.print(tree[i] + " ");
    preorder(2 * i + 1);
    preorder(2 * i + 2);
}

```

- **Pros:** no references (less memory), and the nodes sit next to each other in memory (cache friendly). Parent and child are one arithmetic step apart.
- **Cons:** only works well for **complete** trees. A skewed tree would leave huge gaps in the array.

INSIDE JAVA.UTIL **PriorityQueue** is a complete binary tree in an array

`java.util.PriorityQueue` keeps a **binary min-heap** in `Object[]` `queue`, with the smallest element at index 0 and the children of `i` at `2i+1` and `2i+2`. That is why printing it looks unsorted: you see the array in level order, not sorted order. Output of `JavaTreesDemo.java`:

TEXT

```
PriorityQueue toString()    : [33, 45, 72, 60, 88, 91, 79]    ← the internal
poll() repeatedly         : 33 45 60 72 79 88 91             ← smallest first
```

We build heaps and heap sort on Day 7.

6. Binary search trees

DEFINITION **Binary search tree (BST)**

A binary tree in which, for **every** node, all keys in its **left** subtree are **smaller** than its key, and all keys in its **right** subtree are **larger**.

This one rule means that at every node we can **discard one subtree**, just like binary search on a sorted array discards half of it.

Search — $O(h)$

JAVA `BST.java`

```
public boolean contains(K key) {
    Node cur = root;
```

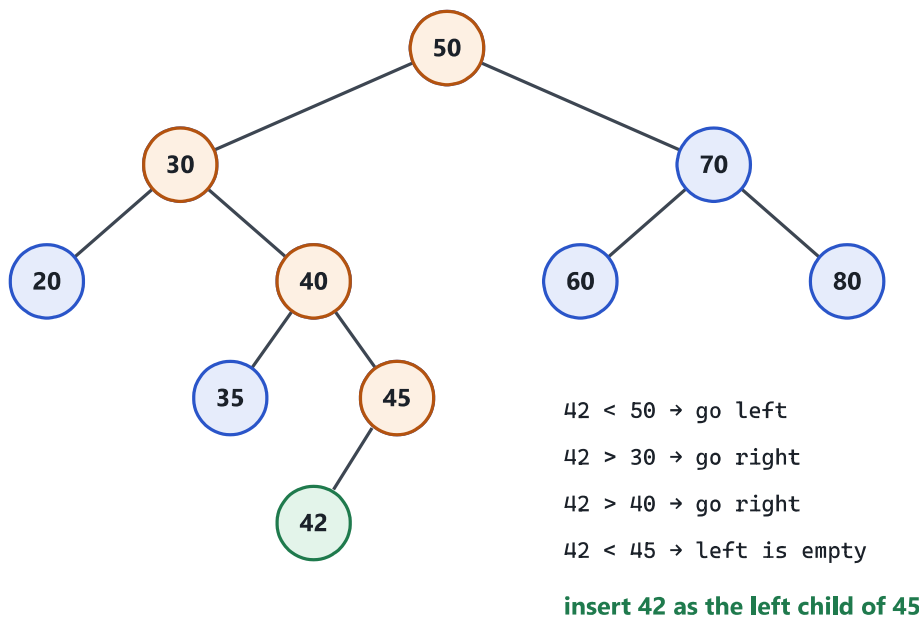
```

while (cur ≠ null) {
    int c = key.compareTo(cur.key);
    if (c == 0) return true;
    cur = c < 0 ? cur.left : cur.right;    // go one way, never both
}
return false;
}

```

Insert — $O(h)$

Search for the key. When you fall off the tree (reach `null`), that empty spot is exactly where the key belongs.



4 comparisons for 10 keys. Work = height of the tree: $O(h)$

Search follows the same path. If the path ends at an empty spot, the key is not present, and that is where insert puts it.

JAVA

```

public void insert(K key) { root = insert(root, key); }

private Node insert(Node n, K key) {
    if (n == null) return new Node(key);    // the empty spot: put it
    int c = key.compareTo(n.key);

```

```
if (c < 0)      n.left = insert(n.left, key);
else if (c > 0) n.right = insert(n.right, key);
// c == 0: duplicate, ignored (like a Set)
return n;                                     // return the (unchanged)
}
```

TIP The "return the new subtree" pattern

`n.left = insert(n.left, key)` looks odd at first. It lets the recursive call **replace** the child: `null` becomes a new node on insert, and a node becomes `null` (or its child) on delete. The same pattern returns in AVL rotations tomorrow.

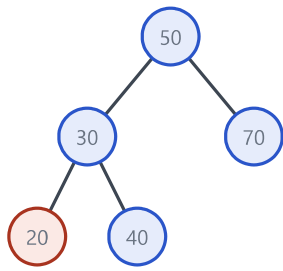
Minimum, maximum and sorted order

- **Minimum:** keep going **left**. **Maximum:** keep going **right**.
- An **in-order** traversal of a BST visits the keys in **sorted** order: `[20, 30, 35, 40, 45, 50, 60, 65, 70, 80]`.

WATCH OUT The classic wrong BST check

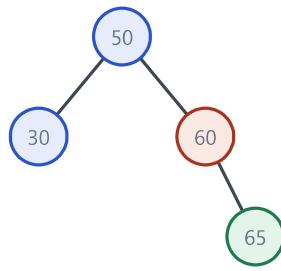
Checking only `left.key < node.key < right.key` for each parent–child pair is **not enough**. In the tree `10 → left 5 → right 12`, every pair looks fine, but 12 sits in the **left** subtree of 10. The correct check passes an allowed range down the tree:
`isBST(n, min, max)`. See `TreeTypesCheck.java`.

Delete — three cases



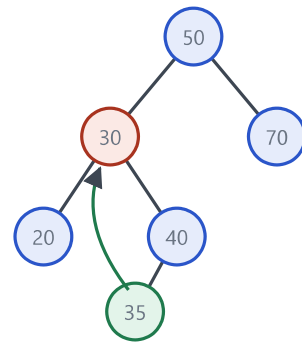
① leaf: delete 20

just remove it
parent.left = null



② one child: delete 60

the child takes its place
50.right = 65



③ two children: delete 30

copy the in-order successor (35),
then delete 35 from the right subtree

Case ③ turns into case ① or ②: the successor never has a left child

JAVA

```
private Node delete(Node n, K key) {
    if (n == null) return null; // not found
    int c = key.compareTo(n.key);
    if (c < 0) { n.left = delete(n.left, key); return n; }
    if (c > 0) { n.right = delete(n.right, key); return n; }
    // found it
    if (n.left == null && n.right == null) return null; // ① leaf
    if (n.left == null) return n.right; // ② only a right
    if (n.right == null) return n.left; // ② only a left c
    Node succ = n.right; // ③ two children:
    while (succ.left != null) succ = succ.left; // smallest key
    n.key = succ.key; // copy it up
    n.right = delete(n.right, succ.key); // delete it be
    return n;
}
```

Output of BST.java :

TEXT

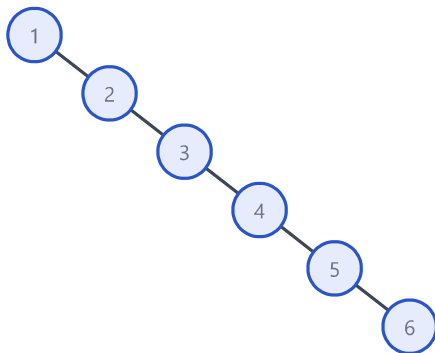
delete 20 (leaf) → [30, 35, 40, 45, 50, 60, 65, 70, 80]
 delete 60 (one child) → [30, 35, 40, 45, 50, 65, 70, 80]
 delete 30 (two children) → [35, 40, 45, 50, 65, 70, 80] (35, the successor, t
 delete 50 (the root) → [35, 40, 45, 65, 70, 80]

**QUICK
CHECK**

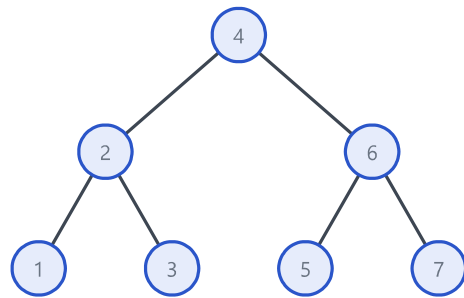
In case ③, could we use the in-order predecessor instead of the successor?

Show answer

Cost — it all depends on the height



insert 1, 2, 3, 4, 5, 6 → height 5



insert 4, 2, 6, 1, 3, 5, 7 → height 2

20,000 keys: sorted → height 19,999 · random → height 37

A BST's speed depends on its shape. Its shape depends on the order of insertion.

Operation	Balanced BST ($h \approx \log n$)	Degenerate BST ($h = n - 1$)
search / insert / delete	$O(\log n)$	$O(n)$
min / max	$O(\log n)$	$O(n)$
in-order traversal	$O(n)$	$O(n)$

Output of `DegenerateBST.java` for 20,000 keys:

Insertion order	Height	Build time	Steps to find the largest key
random	37	6 ms	13
sorted	19,999	331 ms	20,000

A plain BST gives **no guarantee**. Real data (roll numbers, dates, IDs) often arrives already sorted, which is the worst case. The fix, **self-balancing** trees, comes tomorrow.

7. Trees inside `java.util`

INSIDE JAVA.UTIL `TreeMap` and `TreeSet` : a balanced BST you can use today

`TreeMap<K, V>` is a **red-black tree**, a BST that rebalances itself on every insert and delete, so its height stays $O(\log n)$ whatever the input order. `TreeSet<E>` is a `TreeMap` with dummy values. Everything a BST can do comes as a ready method:

TEXT

```
TreeSet (in-order = sorted) : [33, 45, 60, 72, 79, 88, 91]
floor(80)   largest ≤ 80   : 79
ceiling(80) smallest ≥ 80  : 88
headSet(60) (< 60)         : [33, 45]
```

- `get`, `put`, `remove`, `floorKey`, `ceilingKey`, `firstKey` and `lastKey` are all **$O(\log n)$** .
- Keys must be `Comparable`, or you pass a `Comparator`. That is our `compareTo` from `BST.java`.
- Use `TreeMap` when you need **sorted keys or range queries**. Otherwise `HashMap` (Day 8) is faster, at $O(1)$ on average.

Need	Class	Structure inside
sorted set or map, range queries	<code>TreeSet</code> , <code>TreeMap</code>	red-black BST

Need	Class	Structure inside
repeatedly take the smallest item	<code>PriorityQueue</code>	binary heap in an array (complete tree)
fast lookup, no ordering	<code>HashSet</code> , <code>HashMap</code>	hash table (Day 8)

8. Summary

- **Tree**: a root, parent–child edges, no cycles, $n - 1$ edges. **Binary tree**: at most 2 children per node.
- Terms: root, leaf, internal node, siblings, subtree, depth, level, height (in edges; -1 for an empty tree).
- Types: full, **complete** (array-friendly), perfect ($2^{h+1} - 1$ nodes), **balanced** (height $O(\log n)$), degenerate (a list), **BST**.
- Traversals: pre-order (Root L R), **in-order** (L Root R, sorted for a BST), post-order (L R Root), and **level-order with a queue**. DFS can use an explicit stack instead of recursion.
- **Complete tree in an array**: left child $2i+1$, right child $2i+2$, parent $(i-1)/2$.
`PriorityQueue` uses this.
- **BST**: search, insert and delete in **$O(h)$** . Delete has 3 cases, and case ③ uses the in-order successor. Sorted input degrades it to $O(n)$.
- `TreeMap` / `TreeSet` = a self-balancing (red-black) BST with a guaranteed $O(\log n)$.

9. Lab sheet — Day 5

The code files for these tasks will be shared by your faculty. Run a file directly with `java FileName.java` (Java 11+), or open it in your IDE.

LAB Task 1 — Binary search tree with operations (syllabus lab)

Write a program that implements a BST with these operations:

- **Create()**: build the tree from a list of keys

- **Tree traversals: BFS, DFS, PostOrder, InOrder, PreOrder**
- **Delete()**: handle all three cases

Use `BSTLab.java` as the reference menu. Type your own version. Test deleting a leaf, a node with one child, a node with two children, the root, and a key that is not present.

LAB Task 2 — Recursive tree properties

Add these to your BST and state the Big O of each: `size()`, `height()`, `countLeaves()`, `sumOfKeys()`, `kthSmallest(k)` (hint: in-order with a counter), and `printLevels()`, which prints one line per level.

LAB Task 3 — Iterative traversals

Implement pre-order, in-order and post-order **without recursion**, using `ArrayDeque` as a stack. Run them on a degenerate tree of 10 lakh nodes and confirm that there is no `StackOverflowError`.

LAB Task 4 — Complete tree in an array

Store the keys `A` to `J` in a `char[]` as a complete tree. Print each node's parent, left child and right child using the formulas, then print pre-order, in-order and post-order using indexes only.

LAB Task 5 — Watch the shape change

Insert 1 to 15 in sorted order, then in the order `8 4 12 2 6 10 14 1 3 5 7 9 11 13 15`. Print the height and the tree sideways each time. Then run `DegenerateBST.java`.

TRY IT Home challenge

1. Check whether a binary tree is a valid BST, using the min/max range method.
2. Find the **lowest common ancestor** of two keys in a BST in $O(h)$.
3. Rebuild a binary tree from its pre-order and in-order sequences.

10. Quick quiz

QUICK
CHECK

1. Insert 40, 20, 60, 10, 30, 50, 70 into an empty BST. What is the post-order traversal?

Show answer

QUICK
CHECK

2. In an array-stored complete binary tree, which indexes hold the children and the parent of index 5?

Show answer

QUICK
CHECK

3. Which traversal would you use to delete every node of a tree safely, and why?

Show answer

QUICK
CHECK

4. You delete a node with two children from a BST, replacing it with its in-order successor. Why can the successor never have a left child?

Show answer

QUICK
CHECK

5. Why does `TreeMap.get()` guarantee $O(\log n)$ while our `BST.contains()` does not?

Show answer

Next class — Day 6: AVL Trees & Searching. We learn rotations that keep a BST balanced automatically, and then sequential search and binary search, with a full analysis of both.